```kotlin
fun main() {  new *
    for (i in 1 ≤ .. ≤ 10) {
        println("Hello, welcome to the tutorial")
    }
    TODO( reason: "Write some cool code so that they are impressed")
}
```

Programming Tutorial - 27

7. Tutorial - Exceptions & Interfaces

Kuzey Aydınhan | 09. January 2026

Contents

1. Overview

2. Exceptions

3. Interfaces

Information

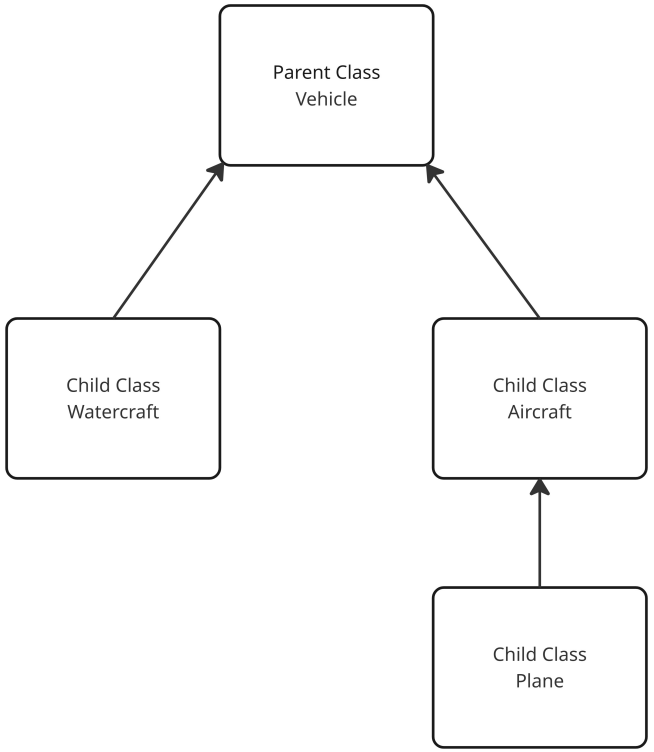
- Homework 4 deadline: 15. Jan.
- Next week will do revision on topics for Präsenzübung
  - You can write any topic suggestions in ILIAS-Forum or by email
- Dictionaries are allowed in Präsenzübung

Important

The Präsenzübung is on 21. Jan.
If you want to use a dictionary, you have to register it until 14. Jan.

Last Time

Inheritance



- Prevents code duplication
- Keyword: **`extends`**
- Transitive relation

Last Time

Inheritance

```java
public class Aircraft {
    String aircraftId;

    void fly() {
        // ...
    }
}

public class Plane extends Aircraft {
    Flaps flaps;

    void controlFlaps() {
        // ...
    }

    public static void main(String[] args) {
        Plane plane = new Plane();

        plane.fly();  // Inherited
        plane.aircraftId;  // Inherited
        plane.controlFlaps();  // Own method
    }
}

```

It will inherit:

- Attributes
- Methods
- Parent classes
- **No** constructors!!

Last Time

Dynamic Binding

- Dynamic data type decides which method to be called

```java
public class Animal {
    public void speak() {
        System.out.println("Animal speaks");
    }
}

public class Dog extends Animal {
    @Override
    public void speak() {
        System.out.println("Dog barks");
    }
}

```

```java
public static void main(String[] args) {
    Animal foo = new Dog();
    foo.speak();  // output: "Dog barks"
}

```

- `Animal` is the **static type** for `foo`
- `Dog` is the **dynamic type** for `foo`

Last Time

Abstract Classes

```java
public abstract class Animal {
    public abstract void makeSound();
}

public class Dog extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Woof");
    }
}

```

- Abstract classes provide method signatures to be implemented
- These classes can't be initialized
- This way specify the base methods of classes

Homework

UI/IO

- Don't create an IO class to call from everywhere
- There shouldn't be any I/O action in a **model** class
  - i.e. don't print the game tiles in the `Game` class
  - have a UI class to run the loop and do the input/output stuff

```java
class ConnectFourUI {
    void start() {
        scanner = new Scanner(System.in);
        System.out.println(game.getBoard());

        while (isRunning) {
            handleMove();
        }
        scanner.close();
    }

    void handleMove() {
        String input = scanner.nextLine();
        // Check command, parse command and print output
        // ...
    }
}

```

```java
class Game {
    private Board board;
    private List<Player> players;
    private int currentPlayer;
    private int moveCount;
    private int winner;

    // getter, setter methods
    // ...

    void printGame() {
        // DONT DO THIS
    }
}

```

Guideline

- https://sdq.kastel.kit.edu/programmieren/IO/UI

Motivation

- Exceptions
- Interfaces

Exceptions

Idea

- There can be unexpected problems
  - Alters the program
  - Problem must be resolved before continuing → otherwise program is ended

- Usual process:
  - A problem occurs
  - Continuing is not possible
  - Locally handling the error is not meaningfully possible
  - Problem needs to be handled elsewhere

Exceptions

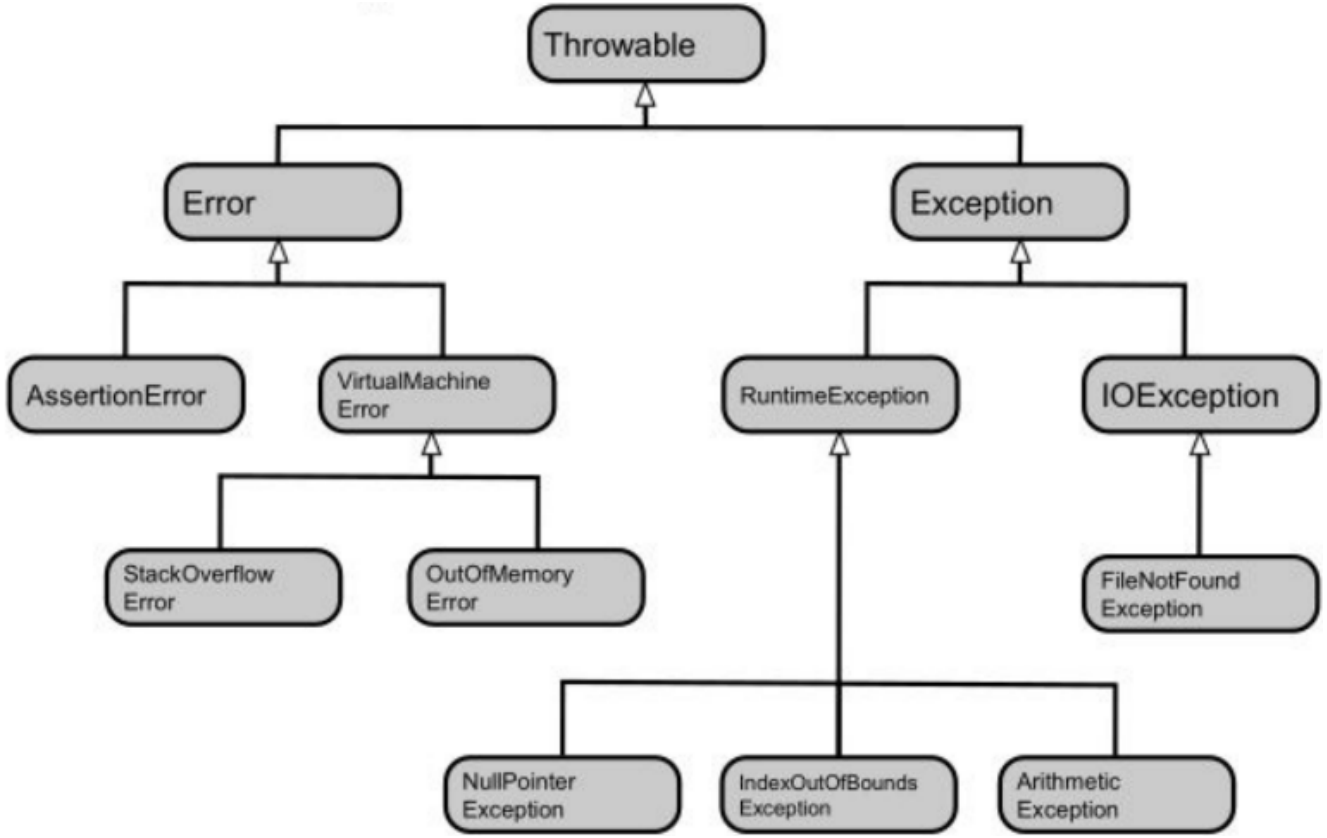
Java

- Exceptions are `Objects` implementing `Throwable`
  - Instantiation with `new`
  - Throwing with `throw`
- Two constructors:
  - Default constructor
  - `String` parameter
- `exception.getMessage()` → get information

```java
class Main {
    public static void main(String[] args) {
        readFile(args[0]);
    }

    private static void readFile(String path) throws FileNotFoundException {
        if (!isValid(path)) {
            throw new FileNotFoundException("Path is not valid");
        }

        // Read file
        // ...
    }
}

```

Exceptions

Hierarchy



Errors vs Exceptions

Errors:

- Important problems
  - i.e. out of memory
- Should never be handled in code

Exceptions:

- Handleable program mistakes
  - i.e. file not found
  - Null pointer
  - ...
- Mostly discovered in execution

Exceptions

Try-Catch Block

```java
try {
    // Code that can cause exception
} catch ( /* Exception to be caught */ ) {
    // Handling the exception
}
// Continue with normal code

```

Example:

- `Integer#parseInt` `throws` `NumberFormatException`

```java
void main(String args[]) {
    if (args.length != 1) {
        return;
    }

    int number;
    try{
        number = Integer.parseInt(args[0]);
    } catch (NumberFormatException e) {
        System.err.println("Error: " + e.getMessage());
    }
}

```

Exceptions

What NOT to do

- (Almost) never handle runtime exceptions
- Don't catch the unspecific `Exception` class
- Never put try-catch block all around the program
- Avoid empty catch blocks
- Don't use exceptions as control flows

Runtime Exceptions

Important

`RuntimeException` and its sublacesses should not be handled in general

- If there is `RuntimeException` occurring
  → it's caused by a mistake on coder's end
  → fix the code don't handle the exception

BAD

```
String maybeNull = null;
boolean result;
try {
    result = maybeNull.equals("NotNull");
} catch (NullPointerException) {
    return false;
}
return result;

```

- BUT you can handle the `NumberFormatException` and only this exception

GOOD

```
int number;
try {
    number = Integer.parseInt(input);
} catch (NumberFormatException e) {
    System.err.println("Error: " + e.getMessage());
}

```

Guideline : https://sdq.kastel.kit.edu/programmieren/Runtime_Exceptions

JavaDoc

- Always inlcude exception in `JavaDoc`
- Inform other developers of the occurring exceptions
- Keyword : **`@throws`**

```java
/**
 * Method description
 *
 * @throws FileNotFoundException if the entered file path is faulty
 */
 static voide readFile(String path) throws FileNotFoundException {
    if (!isValid(path)) {
        throw new FileNotFoundException("Path is not valid");
    }
    // ...
 }

```

Too Generic Catching

- If a method is throwing more than one exception
- Don't handle all exceptions by catching a supertype
  - `Exception`
  - `Throwable`
- There is no single way of handling all exceptions:
  - `IOException` : Retry
  - `NullPointerException` : Fix code
  - `IllegalArgumentException` : Error output

BAD

```java
public static void main(String[] args) {
    int parsed;
    try {
        parsed = Integer.parseInt(args[0]);
    } catch (Exception e) {
        System.err.println(e.getMessage());
    }
}

```

GOOD

```java
public static void main(String[] args) {
    if (args.length != 1) {
        return;
    }
    int parsed;
    try {
        parsed = Integer.parseInt(args[0]);
    } catch (NumberFormatException e) {
        System.err.println(e.getMessage());
    }
}

```

Big Try-Catch Blocks

BAD

```java
try {
    if (args.length != 1) {
        return;
    }
    int parsed = Integer.parseInt(args[0]);
    // ...
} catch (Exception e) {
    System.err.println(e.getMessage());
}

```

GOOD

```java
// ...
int parsed;
try {
    parsed = Integer.parseInt(args[0]);
} catch (NumberFormatException e) {
    System.err.println(e.getMessage());
}
// ...

```

- Don't put the try-catch block around whole code
- Put it only where necessary

Guideline

https://sdq.kastel.kit.edu/programmieren/

Exception Control Flow

- Exceptions should not be used to control program flow
- Use `Optional<T>` or `null` returns instead

BAD

```java
try {
    if (args.length != 1) {
        throw new IllegalArgumentException();
        // skip to catch to control the flow
    }
    int parsed = Integer.parseInt(args[0]);
    // ...
} catch (Exception e) {
    System.err.println(e.getMessage());
    return;
}

```

GOOD

```java
if (args.length != 1) {
    System.err.println(...);
    return;
}
int parsed;
try {
    parsed = Integer.parseInt(args[0]);
} catch (NumberFormatException e) {
    System.err.println(e.getMessage());
}

```

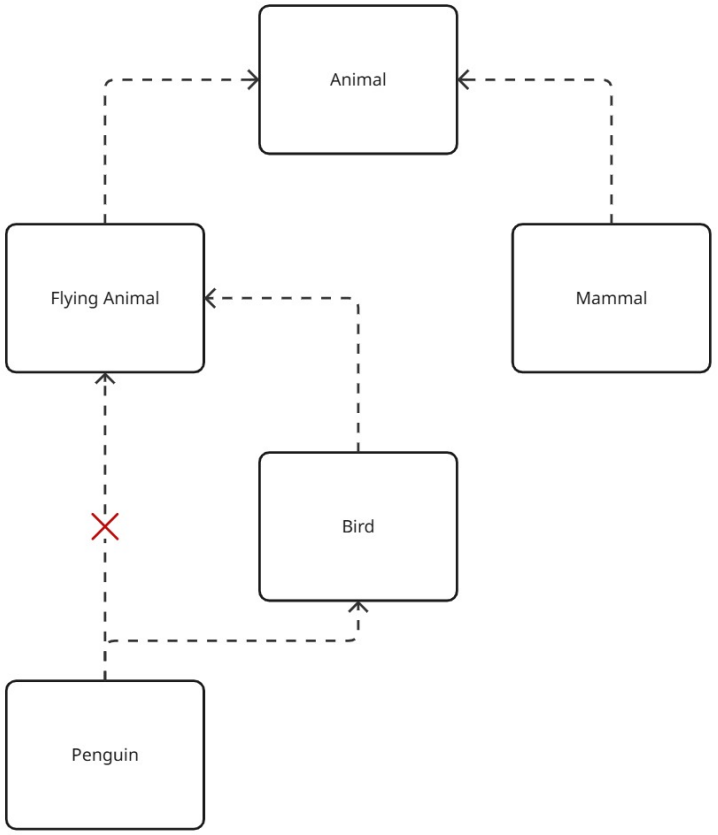
Guideline

https://sdq.kastel.kit.edu/programmieren/Exceptions_Kontrollfluss

Interfaces

Problem

- A class can’t extend multiple classes
- It gets hard/complicated to extend classes
- What if a class can be similar to two other superclasses at the same time?
- How to model "can" relations



Interfaces

```java
interface CanFly {
    public void fly();
}

interface CanWalk {
    public void walk();
}

class Penguin extends Bird implements CanWalk {
    @Override
    public void walk() {...}

    @Override
    public String chirp() {...}
    // ...
}

```

- Used for modeling **"can"** relations
- Defines abilities (method signatures)
- Will be **`implemented`** by classes
  → not extended!

- All methods defined will be **public**

Interfaces

Multiple Inheritance

- A class can implement multiple interfaces
- The class must implement all methods from all implemented interfaces

```java
interface CanFly {
    public void fly();
}

interface CanWalk {
    public void walk();
}

class Raven extends Bird implements CanWalk, CanFly {
    @Override
    public void fly() {...}

    @Override
    public void walk() {...}

    @Override
    public String chirp() {...}
    // ...
}

```

Interfaces

Instantiation

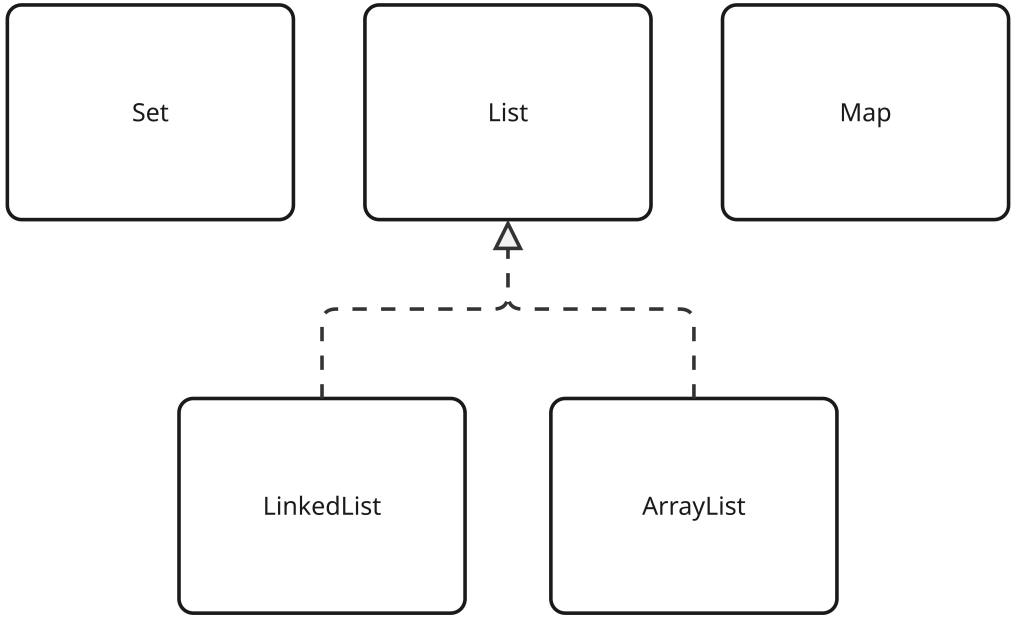
```
interface CanFly {
    public void fly();
}

public static void main(String[] args) {
    // NOT POSSIBLE
    CanFly fly = new CanFly();
}

```

- Interfaces cannot be instantiated

Interfaces

Types

- Interfaces can be used for creating "types"
- A class that implements an interface is a "subtype"
- Objects can have multiple types



Exercise 1

- Think of and model two class implementations:
  - one that extends `Edible`
  - one that extends `Fruit`
- Then implement the method `calculateCalories` → returns sum of calories of edibles

```java
public interface Edible {
    public int getCalories();
    public double getProteinValue();
    public double getCarbohydrateValue();
}

public interface Fruit extends Edible { }

final class CalorieCalculator {
    private CalorieCalculator() { }

    public static int calorieCalculator(Edible[] edibles) {
        // TODO: implement me
    }
}

```

Abstract vs. Interface

Abstract Class

- Class can only extend single abstract class
- Can have attributes and method bodies (base implementation)
- Contains partial implementation for common functionality

Interface

- Class can implement multiple interfaces
- Can't have attributes and method bodies
- Only creates an abstraction of the usage

Principle of Least Assumption

- When defining there is a choice: Class or Interface
- The Principle states:
  - We use the class, only if the interface methods are not enough!
  - Most commonly is the interface preferred
  - Use the most simple interface that is enough, out of all compatible interfaces
  - For example: `Collection` instead of `List`, `List` instead of `ArrayList`
- In general: Use the most general type possible

```java
public static void main(String[] args) {
    // Not LinkedList -> List<String>
    List<String> list = new LinkedList<>();
    list.add("A"); list.add("B"); list.add("C");

    // Not ListIterator -> Iterator<String>
    Iterator<String> itr = list.iterator();
    while (itr.hasNext()) {
        String element = itr.next();
        System.out.print(element + " ");
    }
}
```

Guideline

- https://sdq.kastel.kit.edu/programmieren/

Exercise 2

- Implement the concrete classes: **`EmailNotification`** and **`EmailNotificationRepository`**

```java
interface Notification {
    String getRecipient();
    String getMessage();
}

interface NotificationRepository {
    boolean save(Notification notification);
    Notification findByRecipient(String recipient);
}

```

```java
class EmailNotification implements Notification {
    @Override
    public String getRecipient() { }

    @Override
    public String getMessage() { }

    public String getEmailRecipient() { }
}

```

```java
class EmailNotificationRepository implements NotificationRepository {
    @Override
    public boolean save(Notification notification) { }

    @Override
    public Notification findByRecipient(String recipient) { }

    public Notification findByEmailRecipient(String emailRecipient) { }
}

```

Conclusion

- Exceptions
- Interfaces

- **Next week: full revision and preparation for Präsenzübung**

Reminder

Präsenzübung on 21. January, dictionary registration until 14. January
Homework 4 deadline: 15. January

Exercise 1

```java
class Apple implements Fruit {
    @Override
    public int getCalories() {
        return 48;
    }

    @Override
    public double getProteinValue() {
        return 1.44;
    }

    @Override
    public double getCarbohydrateValue() {
        return 10.26;
    }
}

```

```java
class Pasta implements Edible {
    @Override
    public int getCalories() {
        return 157;
    }

    @Override
    public double getProteinValue() {
        return 2.58;
    }

    @Override
    public double getCarbohydrateValue() {
        return 20.12;
    }
}

```

Exercise 1

```java
final class CalorieCalculator {
    private CalorieCalculator() { }

    public static int calorieCalculator(Edible[] edibles) {
        int calories = 0;
        for (Edible edible : edibles) {
            calories += edible.getCalories();
        }
        return calories;
    }
}

```

Exercise 2

```java
class EmailNotification implements Notification {
    private final String recipient;
    private final String message;
    private final String recipientMail;

    public EmailNotification(String recipient, String message, String recipientMail) {
        this.recipient = recipient;
        this.message = message;
        this.recipientMail = recipientMail;
    }

    @Override
    public String getRecipient() { return recipient; }

    @Override
    public String getMessage() { return message; }

    public String getRecipientMail() { return recipientMail; }
}

```

Exercise 2

```java
class EmailNotificationRepository implements NotificationRepository {
    private List<EmailNotification> notifications = new ArrayList<>();

    @Override
    public boolean save(Notification notification) {
        if (notification.getClass().equals(EmailNotification.class)) {
            EmailNotification emailNotification = (EmailNotification) notification;
            return notifications.add(emailNotification);
        }
        return false;
    }

    @Override
    public Notification findByRecipient(String recipient) {
        for (Notification notification : notifications) {
            if (Objects.equals(recipient, notification.getRecipient())) { return notification; }
        }
        return null;
    }

    public Notification findByEmailRecipient(String emailRecipient) {
        for (EmailNotification notification : notifications) {
            if (Objects.equals(emailRecipient, notification.getRecipientMail())) { return notification; }
        }
        return null;
    }
}

```